

Assignment 04

Course Code: CS-112

Name: _____

Course Title: Object Oriented Programming

Registration No. _____

Total Marks: 15

Discipline/Semester/Section: BSDS

Assigned Date: 16-05-2025

Deadline: 19-05-2025

Submission Instructions:

- ✓ Plagiarism will result in zero marks.
- ✓ Add comments in each part of the code to explain functionality.
- Submit the assignment in the following format:**
- ✓ Take a screenshot of your Java code, Package Structure and output.
- ✓ Create separate classes (open new java class).
- ✓ Use meaningful class, methods, and variable names etc.
- ✓ Compile everything into a Word file.
- ✓ Submit a hard copy before the mentioned deadline.

(CLO-3, GA-4, C4)

Q1.

A university wants to automate its library system. The system should handle core functionalities such as managing books, users (students and faculty), book borrowing, returning, and generating reports. It should also support advanced features like searching books by category, tracking borrowed books, calculating fines, and providing usage analytics.

You are required to **analyze** the functional requirements of this system using Object-Oriented Programming (OOP) principles. Your analysis should identify how to best utilize appropriate data structures (**Arrays, ArrayLists, HashMaps, packages**) and binding techniques (static and dynamic) to build a scalable and efficient solution.

1. Package Requirements

Create at least **two packages**:

- ✓ **library.models** → for all class definitions like Book, User, Student, Faculty.
- ✓ **library.operations** → for operations like LibraryManager, FineCalculator, etc.

2. Create a class hierarchy:

- ✓ **User** → superclass with common fields like id, name, and methods like viewBorrowedBooks().
- ✓ **Student and Faculty** → subclasses with specific borrowing limits and overriding method behavior (**dynamic binding**).

3. Use static binding:

Implement overloaded methods in **LibraryManager** for:

- ✓ addBook(Book b)
- ✓ addBook(Book[] books)
- ✓ addBook(String title, String author, ...)

4. Use Arrays:

- ✓ Use a **1D array** to maintain a small inventory of reference-only books.
- ✓ Use a **2D array** to simulate a floor-wise book rack ([floor][rack]).

5. Use ArrayList:

- ✓ Maintain an ArrayList<Book> for general book inventory.
- ✓ Implement dynamic addition/removal when books are borrowed or returned.

6. Use HashMap:

- ✓ Maintain a HashMap<User, ArrayList<Book>> to track which user borrowed which books.
- ✓ Use another HashMap<Book, Integer> to count how many times each book was borrowed.

7. Additional Functionalities

Allow users to:

- ✓ Borrow books (with limit: 3 for students, 5 for faculty).
- ✓ Return books.
- ✓ View borrowed books (polymorphic behavior based on user type).

Calculate and apply fine using:

- ✓ A FineCalculator class with rules:
- ✓ Students: RS 100/day after 7 days
- ✓ Faculty: RS 50/day after 14 days

Generate report:

- ✓ List top 3 most borrowed books.
- ✓ Display books by category (e.g., Science, Literature, IT, etc.).

